

# ICS 2207: Scientific Computing

## Capstone Project Brief, Six-Level Incremental Design, Rubric, and Datasets

### Project Title: HydroSense-Kenya

*A Scientific Computing System for Smart Irrigation, Water Balance Simulation, and Climate-Aware Decision Support*

Prepared for Python-based scientific computing instruction, February-May 2026 semester

|                   |                                                                           |
|-------------------|---------------------------------------------------------------------------|
| Language          | Python                                                                    |
| Main Libraries    | NumPy, Pandas, Matplotlib, pytest, optional SciPy for verification only   |
| Project Structure | Six incremental levels, from scientific framing to AI-assisted validation |
| Core Domain       | Smart irrigation, water balance, climate-aware decision support           |
| Assessment Use    | Mark out of 100                                                           |

#### Document Purpose

This document gives the complete student-ready project brief, six-level milestone structure, datasets, marking rubric, code-audit plan, and reproducibility requirements.

# 1. Course Alignment and Rationale

HydroSense-Kenya is designed as a serious scientific-computing project rather than a narrow programming exercise. It aligns with the course emphasis on Python, computational thinking, numerical methods, data-science foundations, simulation, error analysis, reproducible scientific programs, and responsible use of AI-assisted programming tools.

The project follows the intellectual arc of scientific computing: problem framing, data acquisition, mathematical modelling, numerical computation, simulation, optimization, validation, interpretation, and scientific communication.

| Syllabus Area                  | How the Project Tests It                                                                                                        |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Python for scientific work     | Students write reusable Python functions, notebooks, modules, and tests.                                                        |
| Numerical methods              | Students implement root finding, numerical differentiation, numerical integration, and linear systems methods.                  |
| Error and stability            | Students evaluate floating point behaviour, error propagation, convergence, and stability of simulation steps.                  |
| NumPy and vectorization        | Students compare loop-based and vectorized computation for evapotranspiration and water balance calculations.                   |
| Pandas and data cleaning       | Students clean missing values, detect outliers, summarize data, and justify cleaning decisions.                                 |
| Matplotlib visualization       | Students produce interpretable scientific figures for rainfall, soil moisture, water deficit, uncertainty, and recommendations. |
| Simulation and Monte Carlo     | Students simulate uncertain rainfall and soil-moisture evolution under different climate conditions.                            |
| Differential equations         | Students approximate continuous soil-water dynamics using Euler and Runge-Kutta methods.                                        |
| Optimization                   | Students minimize irrigation water use while respecting moisture and crop-stress constraints.                                   |
| AI-assisted programming        | Students may use AI tools, but must document, test, and validate all AI-supported work.                                         |
| Reproducibility and code audit | Students submit a structured repository, requirements file, tests, and reproducible notebooks.                                  |

## 2. Project Scenario

A university demonstration farm, agricultural innovation hub, or smallholder farm wants to improve water-use efficiency. The farm has daily weather records, soil-moisture sensor readings, tank-level measurements, pump-flow data, and crop-zone parameters. Students must build a Python scientific-computing system that estimates soil-water balance, detects water stress, simulates uncertainty, and recommends irrigation decisions.

### 2.1 Central Scientific Question

Given weather and soil-sensor data, how can we model water availability, estimate water deficit, simulate future soil moisture, and recommend an efficient irrigation plan that minimizes water use without exposing crops to moisture stress?

### 2.2 Core Water Balance Model

Students should begin from the simplified discrete water-balance equation:

$$S_{(t+1)} = S_t + R_t + I_t - ET_t - D_t$$

| Term   | Meaning                                         |
|--------|-------------------------------------------------|
| $S_t$  | Soil water storage or soil moisture at time $t$ |
| $R_t$  | Rainfall contribution at time $t$               |
| $I_t$  | Irrigation applied at time $t$                  |
| $ET_t$ | Evapotranspiration or crop water loss           |
| $D_t$  | Drainage or water loss beyond field capacity    |

## 2.3 Simplified Evapotranspiration Estimate

For teaching purposes, students may use a simplified empirical estimate of daily evapotranspiration. The formula is not meant to replace a full agronomic model, but it gives students a computationally meaningful relationship among temperature, humidity, wind, and solar intensity.

$$ET = \max(0, 0.12 \cdot T + 0.35 \cdot W + 2.4 \cdot \text{Solar} - 0.025 \cdot H)$$

## 3. Student Brief

Each group will build HydroSense-Kenya as an incremental scientific computing project. Each level must be completed before moving to the next. The final submission should show a clear progression from a simple computational experiment to a tested, reproducible, and defensible decision-support model.

| Final Product Component     | Expected Output                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| Scientific notebook         | A clean, executable Jupyter or Colab notebook integrating the full workflow.                              |
| Python modules              | Functions for data cleaning, numerical methods, simulation, optimization, and visualization.              |
| Datasets                    | Raw and cleaned datasets, including a clear record of data-quality issues and cleaning decisions.         |
| Numerical methods           | Manual implementation of required methods and comparison with trusted reference results where applicable. |
| Simulation and optimization | A model that simulates soil moisture and proposes an irrigation schedule.                                 |
| Scientific report           | A concise but rigorous report explaining assumptions, methods, results, limitations, and recommendations. |
| AI-use log                  | A transparent record of how AI tools were used and how outputs were verified.                             |
| Presentation                | A short scientific presentation with demonstration, interpretation, and code-audit defense.               |

## 4. Six-Level Incremental Project Structure

The six levels are cumulative. Level 6 must integrate and validate the outputs produced in Levels 1 to 5.

### Level 1: Scientific Problem Framing and Python Foundation

**Main goal:** Translate a real irrigation problem into a computational problem using Python.

**Concepts tested:**

- Scientific computing foundations
- Computational thinking
- Python variables, expressions, conditionals, loops, and functions
- Reproducibility and computational ethics

**Required tasks:**

1. Define the scientific problem in a Kenyan or African smart-agriculture context.
2. Load or generate the initial datasets and inspect their structure.
3. Create a data dictionary for all variables.
4. Write reusable Python functions for evapotranspiration and water balance.
5. Create at least one basic plot of rainfall or soil moisture over time.
6. State the assumptions and limitations of the initial model.

**Deliverables:**

- Level\_1\_Problem\_Framing.ipynb
- Data dictionary
- One basic scientific plot

- Problem statement of 500 to 700 words

**Evidence of competence:** The group can move from a real-world problem to a computational representation.

## Level 2: NumPy, Vectorization, Floating Point Errors, and Numerical Reliability

**Main goal:** Demonstrate efficient numerical computation and awareness of numerical error.

**Concepts tested:**

- Lists versus NumPy arrays
- Vectorization
- Performance comparison
- Floating point arithmetic
- Rounding and truncation error
- Error propagation

**Required tasks:**

1. Compute evapotranspiration using ordinary Python loops.
2. Compute the same values using NumPy vectorization.
3. Compare execution time between loop-based and vectorized computation.
4. Demonstrate floating point behaviour using examples such as  $0.1 + 0.2$ .
5. Run an error propagation experiment showing how measurement noise affects irrigation recommendations.
6. Discuss why numerical reliability matters in scientific computing.

**Deliverables:**

- Level\_2\_Vectorization\_and\_Error.ipynb
- Timing comparison table
- Error propagation plot
- Short numerical reliability discussion

**Evidence of competence:** The group understands that scientific computing is not only about getting numbers, but also about reliable computation.

## Level 3: Core Numerical Methods Engine

**Main goal:** Implement core numerical methods from scratch and apply them to irrigation decisions.

**Concepts tested:**

- Bisection method
- Newton-Raphson method
- Secant method
- Convergence analysis
- Finite differences
- Trapezoidal and Simpson integration
- Gaussian elimination or LU decomposition
- Computational complexity

**Required tasks:**

1. Use root finding to calculate the irrigation amount needed to reach target soil moisture.
2. Implement bisection, Newton-Raphson, and secant methods manually.
3. Compare iterations, final error, and convergence status across methods.
4. Estimate the rate of soil-moisture change using forward, backward, and central differences.
5. Estimate cumulative water deficit using trapezoidal and Simpson rules.
6. Solve a three-zone water-allocation problem as  $Ax = b$  using Gaussian elimination or LU decomposition.

**Deliverables:**

- src/numerical\_methods.py
- Level\_3\_Numerical\_Methods.ipynb
- Convergence comparison table
- Integration comparison table
- Linear-system solution and interpretation

**Evidence of competence:** The group can implement, compare, and defend numerical methods rather than merely calling libraries.

## Level 4: Data Cleaning, Scientific Data Analysis, and Visualization

**Main goal:** Use Pandas and Matplotlib to transform raw scientific data into interpretable evidence.

**Concepts tested:**

- Pandas dataframes
- Missing data
- Outlier detection
- Statistical summaries
- Scientific visualization
- Graph interpretation
- Multi-plot and optional 3D visualization

**Required tasks:**

1. Load all datasets using Pandas.
2. Identify missing values, outliers, inconsistent units, and sensor anomalies.
3. Clean the dataset and document every decision.
4. Compute descriptive statistics for key variables.
5. Create at least five scientific visualizations.
6. Interpret each visualization using scientific reasoning, not just visual description.

**Deliverables:**

- Level\_4\_Data\_Analysis\_and\_Visualization.ipynb
- Cleaned dataset
- Five or more scientific visualizations
- Interpretation notes for every visualization

**Evidence of competence:** The group can use data to produce scientific insight and decision-relevant interpretation.

## Level 5: Simulation, Monte Carlo Modelling, Differential Equations, and Optimization

**Main goal:** Build a predictive model and use it to support optimized irrigation decisions.

**Concepts tested:**

- Simulation
- Random number generation
- Monte Carlo methods
- Stochastic modelling
- Euler method
- Runge-Kutta method
- Stability considerations
- Gradient descent or numerical minimization
- Model fitting

**Required tasks:**

1. Simulate soil moisture over 30 days using Euler method.
2. Repeat the simulation using Runge-Kutta method and compare with Euler.
3. Generate rainfall uncertainty scenarios using Monte Carlo simulation.
4. Estimate probability of water shortage, over-irrigation, expected irrigation demand, and worst-case demand.
5. Design an irrigation schedule that minimizes water use while keeping moisture above the minimum threshold.
6. Explain trade-offs between water conservation, crop stress, and pump-energy demand.

**Deliverables:**

- Level\_5\_Simulation\_and\_Optimization.ipynb
- Soil-moisture simulation results
- Monte Carlo uncertainty analysis
- Optimized irrigation schedule
- Scientific explanation of trade-offs

**Evidence of competence:** The group can move from computation to decision intelligence.

## Level 6: AI-Assisted Scientific Programming, Reproducibility, Testing, and Presentation

**Main goal:** Demonstrate responsible AI use, reproducible workflows, validation, and scientific communication.

**Concepts tested:**

- Responsible AI-assisted programming
- Prompt engineering for scientific tasks
- Testing and debugging AI-generated code
- Validation of computational correctness
- Reproducible research
- Peer review and code audit

**Required tasks:**

1. Use an AI tool for at least two specific support tasks, such as generating test cases or improving documentation.
2. Maintain an AI\_USE\_LOG.md file showing prompts, outputs, modifications, and validation methods.
3. Write tests for numerical methods, simulation functions, and data-cleaning functions.
4. Create a reproducible folder structure with README.md and requirements.txt.
5. Prepare a final scientific report and class presentation.
6. Defend the code during a live code audit and reproducibility check.

**Deliverables:**

- AI\_USE\_LOG.md
- tests/ folder
- README.md
- requirements.txt
- Final notebook
- Final scientific report
- Presentation slides

**Evidence of competence:** The group can use AI responsibly while remaining scientifically accountable for correctness.

## 5. Minimum Technical Implementation Requirements

| Component | Minimum Requirement                                                                                                              |
|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| Functions | At least 12 reusable functions distributed across data cleaning, numerical methods, simulation, optimization, and visualization. |

| Component       | Minimum Requirement                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------|
| Root finding    | Bisection, Newton-Raphson, and secant methods implemented manually.                                   |
| Differentiation | Forward, backward, and central finite differences.                                                    |
| Integration     | Trapezoidal rule and Simpson rule.                                                                    |
| Linear systems  | Gaussian elimination or LU decomposition implemented manually, with NumPy used only for verification. |
| Data cleaning   | Explicit treatment of missing values, outliers, and sensor faults.                                    |
| Visualization   | At least five well-labelled scientific plots, each with interpretation.                               |
| Simulation      | Soil-moisture simulation using Euler and Runge-Kutta methods.                                         |
| Monte Carlo     | At least 1,000 rainfall scenarios or a justified alternative number.                                  |
| Optimization    | A defensible algorithm that minimizes irrigation while respecting crop-stress constraints.            |
| Testing         | At least eight automated tests using pytest or equivalent.                                            |
| Reproducibility | README.md, requirements.txt, clear folder structure, and executable notebooks.                        |

## 6. Recommended Project Folder Structure

```

HydroSense-Kenya/
|-- data/
|   |-- raw/
|   |   |-- weather_daily.csv
|   |   |-- soil_sensor_data.csv
|   |   |-- crop_zone_parameters.csv
|   |-- processed/
|       |-- cleaned_irrigation_dataset.csv
|
|-- notebooks/
|   |-- Level_1_Problem_Framing.ipynb
|   |-- Level_2_Vectorization_and_Error.ipynb
|   |-- Level_3_Numerical_Methods.ipynb
|   |-- Level_4_Data_Analysis_and_Visualization.ipynb
|   |-- Level_5_Simulation_and_Optimization.ipynb
|   |-- Level_6_Final_Integration.ipynb
|
|-- src/
|   |-- data_cleaning.py
|   |-- numerical_methods.py
|   |-- simulation.py
|   |-- optimization.py
|   |-- visualization.py
|
|-- tests/
|   |-- test_root_finding.py
|   |-- test_integration.py
|   |-- test_linear_systems.py
|   |-- test_simulation.py
|
|-- reports/
|   |-- final_scientific_report.pdf
|   |-- presentation_slides.pdf
|
|-- AI_USE_LOG.md
-- README.md

```

|-- requirements.txt

## 7. Responsible AI Use Requirement

Students may use AI-assisted coding tools, but AI output must not be treated as authoritative. Every AI-supported function, formula, visualization, or explanation must be inspected, tested, and validated. The project should reward responsible verification, not blind automation.

| Prompt Used                                         | AI Output Summary                | Accepted? | Modified?                    | Validation Method                                      |
|-----------------------------------------------------|----------------------------------|-----------|------------------------------|--------------------------------------------------------|
| Example: Generate pytest cases for bisection method | AI proposed tests for $x^2 - 4$  | Partly    | Tolerance adjusted to $1e-5$ | Compared with known root $x = 2$                       |
| Example: Explain central difference accuracy        | AI gave a conceptual explanation | Yes       | Edited for clarity           | Checked against lecture notes and numerical experiment |
| Student entry 1                                     |                                  |           |                              |                                                        |
| Student entry 2                                     |                                  |           |                              |                                                        |

## 8. Marking Rubric

The project may be marked out of 100.

| Level   | Area Assessed                                                                    | Marks |
|---------|----------------------------------------------------------------------------------|-------|
| Level 1 | Problem framing, Python foundations, assumptions, data dictionary                | 10    |
| Level 2 | Vectorization, performance comparison, numerical error analysis                  | 15    |
| Level 3 | Root finding, differentiation, integration, linear systems, convergence analysis | 20    |
| Level 4 | Pandas data cleaning, statistical summaries, scientific visualization            | 15    |
| Level 5 | Simulation, Monte Carlo analysis, differential-equation modelling, optimization  | 25    |
| Level 6 | AI-use validation, testing, reproducibility, report, presentation, code audit    | 15    |
| Total   |                                                                                  | 100   |

### 8.1 Detailed Grading Descriptors

| Score Band | Descriptor                                                                                                                                         |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 80-100     | Excellent scientific computing work. Correct numerical methods, strong interpretation, clean code, reproducible workflow, and rigorous validation. |
| 70-79      | Very good work with minor weaknesses in interpretation, robustness, testing, or documentation.                                                     |
| 60-69      | Competent implementation but some numerical reasoning, code quality, or data interpretation is shallow.                                            |
| 50-59      | Basic completion with significant gaps in method explanation, testing, or reproducibility.                                                         |
| Below 50   | Incomplete or unreliable work, weak scientific reasoning, major implementation defects, or poor academic integrity.                                |

## 9. Timeline

| Course Period | Project Activity                                         |
|---------------|----------------------------------------------------------|
| Weeks 1-2     | Level 1: problem framing and Python foundations          |
| Weeks 3-4     | Level 2: NumPy, vectorization, and error analysis        |
| Weeks 5-7     | Level 3: numerical methods engine                        |
| Weeks 8-9     | Level 4: data analysis and visualization                 |
| Weeks 10-12   | Level 5: simulation, Monte Carlo, ODEs, and optimization |



| Course Period | Project Activity                                                                     |
|---------------|--------------------------------------------------------------------------------------|
| Weeks 13-14   | Level 6: AI-assisted validation, reproducibility, final presentation, and code audit |

## 10. Final Student Submission Checklist

| Submission Item                  | Required? |
|----------------------------------|-----------|
| Final integrated notebook        | Yes       |
| Python source files              | Yes       |
| Raw datasets                     | Yes       |
| Cleaned dataset                  | Yes       |
| Scientific visualizations        | Yes       |
| Numerical method implementations | Yes       |
| Simulation model                 | Yes       |
| Optimization results             | Yes       |
| Automated tests                  | Yes       |
| AI-use log                       | Yes       |
| Final report                     | Yes       |
| Presentation slides              | Yes       |
| README file                      | Yes       |
| requirements.txt                 | Yes       |

## 11. Dataset Pack Overview

The following synthetic datasets are included. They are intentionally realistic but not perfect. Missing values, sensor faults, and outliers are included so that students must perform real data cleaning rather than assuming ideal data.

| File                     | Purpose                                                               | Important Learning Use                                                                |
|--------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| weather_daily.csv        | Daily rainfall, temperature, humidity, wind speed, and solar index.   | Evapotranspiration, water balance, missing values, outlier handling, visualization.   |
| soil_sensor_data.csv     | Daily noon readings for three farm zones.                             | Sensor-data cleaning, zone comparison, pump-flow analysis, tank-level interpretation. |
| crop_zone_parameters.csv | Crop and zone-specific moisture thresholds and drainage coefficients. | Parameterization, simulation, optimization, group customization.                      |

### 12.1 Recommended Pandas Loading Code

```
import pandas as pd
```

```
weather = pd.read_csv("data/raw/weather_daily.csv", na_values=["NA", ""])
soil = pd.read_csv("data/raw/soil_sensor_data.csv", na_values=["NA", ""])
params = pd.read_csv("data/raw/crop_zone_parameters.csv", na_values=["NA", ""])
```

```
print(weather.info())
print(soil.info())
print(params.info())
```

## Appendix A: weather\_daily.csv

Blank/NA values are intentional and should be treated as missing values. Extreme rainfall and temperature values are included for outlier analysis.

| date       | rainfall_mm | temperature_c | humidity_pct | wind_speed_mps | solar_index |
|------------|-------------|---------------|--------------|----------------|-------------|
| 2026-03-01 | 3.20        | 23.80         | 69.70        | 2.28           | 0.78        |
| 2026-03-02 | 2.20        | 25.90         | 62.00        | 1.62           | 0.71        |
| 2026-03-03 | 3.00        | 26.80         | 64.40        | 2.11           | 0.75        |
| 2026-03-04 | 1.60        | 27.00         | 64.60        | 2.09           | 0.58        |
| 2026-03-05 | 23.70       | 26.60         | 61.00        | 1.36           | 0.62        |
| 2026-03-06 | 0           | 26.70         | 66.80        | 3.07           | 0.49        |
| 2026-03-07 | 1.30        | 24.90         | 61.20        | 1.83           | 0.72        |
| 2026-03-08 | NA          | 24.90         | 69.60        | 3.10           | 0.65        |
| 2026-03-09 | 0           | 23.60         | 64.00        | 2.12           | 0.72        |
| 2026-03-10 | 18.40       | 22.50         | 74.40        | 2.62           | 0.76        |
| 2026-03-11 | 6.70        | 22.40         | 67.60        | 2.32           | 0.80        |
| 2026-03-12 | 7.70        | 22.00         | 68.30        | 2.70           | 0.69        |
| 2026-03-13 | 0           | 23.40         | 63.50        | 2.01           | 0.70        |
| 2026-03-14 | 0           | 45.80         | 66.10        | 2.20           | 0.57        |
| 2026-03-15 | 8.70        | 24.10         | 67.00        | 1.51           | 0.58        |
| 2026-03-16 | 13.40       | 26.00         | 56.30        | 2.16           | 0.50        |
| 2026-03-17 | 0.30        | 26.30         | 63.40        | 1.69           | 0.57        |
| 2026-03-18 | 7.20        | 26.90         | 65.10        | 2.40           | 0.66        |
| 2026-03-19 | 26.30       | 27.40         | 61.60        | 2.11           | 0.76        |
| 2026-03-20 | 15.20       | 26.90         | 60.00        | 2.47           | 0.80        |
| 2026-03-21 | 8.10        | 25.10         | NA           | 1.91           | 0.78        |
| 2026-03-22 | 0.70        | 24.60         | 63.80        | 1.14           | 0.79        |
| 2026-03-23 | 6.80        | 23.70         | 68.10        | 2.39           | 0.65        |
| 2026-03-24 | 1.90        | 21.40         | 68.60        | 1.36           | 0.62        |
| 2026-03-25 | 1.90        | 22.00         | 69.20        | 1.33           | 0.53        |
| 2026-03-26 | 85.00       | 22.50         | 70.50        | 1.65           | 0.58        |
| 2026-03-27 | 3.20        | 23.10         | 74.60        | 2.72           | 0.59        |
| 2026-03-28 | 0           | 22.40         | 64.10        | 2.38           | 0.77        |
| 2026-03-29 | 0           | 24.80         | 64.30        | 2.48           | 0.75        |
| 2026-03-30 | 0.90        | 25.10         | 65.30        | 1.55           | 0.79        |

## Appendix B: soil\_sensor\_data.csv

The table contains daily noon sensor readings for three farm zones. Some anomalies are deliberate for data-cleaning and validation tasks.

| timestamp        | zone_id | soil_moisture_pct | tank_level_liters | pump_flow_lpm | pump_power_watts | sensor_status |
|------------------|---------|-------------------|-------------------|---------------|------------------|---------------|
| 2026-03-01 12:00 | Zone_A  | 33.20             | 4829              | 18.80         | 437              | OK            |
| 2026-03-01 12:00 | Zone_B  | 34.70             | 4728              | 22.10         | 461              | OK            |
| 2026-03-01 12:00 | Zone_C  | 28.20             | 4515              | 25.70         | 490              | OK            |
| 2026-03-02 12:00 | Zone_A  | 36.10             | 4757              | 16.60         | 411              | OK            |
| 2026-03-02 12:00 | Zone_B  | 32.00             | 4645              | 20.40         | 445              | OK            |
| 2026-03-02 12:00 | Zone_C  | 31.00             | 4519              | 24.60         | 492              | OK            |
| 2026-03-03 12:00 | Zone_A  | 33.70             | 4791              | 17.10         | 438              | OK            |
| 2026-03-03 12:00 | Zone_B  | 34.90             | 4603              | 22.10         | 454              | OK            |
| 2026-03-03 12:00 | Zone_C  | 28.90             | 4552              | 26.00         | 490              | OK            |
| 2026-03-04 12:00 | Zone_A  | 31.80             | 4704              | 17.50         | 444              | OK            |
| 2026-03-04 12:00 | Zone_B  | 35.30             | 4552              | 21.20         | 441              | OK            |
| 2026-03-04 12:00 | Zone_C  | 29.30             | 4419              | 28.80         | 500              | OK            |

# Appendices - Student Datasets for HydroSense-Kenya

| timestamp        | zone_id | soil_moisture_pct | tank_level_liters | pump_flow_lpm | pump_power_watts | sensor_status |
|------------------|---------|-------------------|-------------------|---------------|------------------|---------------|
| 2026-03-05 12:00 | Zone_A  | 33.30             | 4836              | 16.50         | 438              | OK            |
| 2026-03-05 12:00 | Zone_B  | 34.60             | 4616              | 20.70         | 462              | OK            |
| 2026-03-05 12:00 | Zone_C  | 28.30             | 4518              | 28.20         | 506              | OK            |
| 2026-03-06 12:00 | Zone_A  | 34.10             | 4529              | 20.30         | 418              | OK            |
| 2026-03-06 12:00 | Zone_B  | NA                | 4451              | 23.40         | 442              | OK            |
| 2026-03-06 12:00 | Zone_C  | 27.70             | 4423              | 24.80         | 488              | OK            |
| 2026-03-07 12:00 | Zone_A  | 30.60             | 4520              | 17.80         | 430              | OK            |
| 2026-03-07 12:00 | Zone_B  | 31.70             | 4479              | 22.90         | 450              | OK            |
| 2026-03-07 12:00 | Zone_C  | 28.70             | 4348              | 26.40         | 499              | OK            |
| 2026-03-08 12:00 | Zone_A  | 31.40             | 4513              | 16.90         | 430              | OK            |
| 2026-03-08 12:00 | Zone_B  | 29.90             | 4409              | 19.80         | 466              | OK            |
| 2026-03-08 12:00 | Zone_C  | 25.70             | 4247              | 26.10         | 489              | OK            |
| 2026-03-09 12:00 | Zone_A  | 29.10             | 4497              | 19.20         | 419              | OK            |
| 2026-03-09 12:00 | Zone_B  | 31.60             | 4370              | 22.00         | 454              | OK            |
| 2026-03-09 12:00 | Zone_C  | 27.00             | 4203              | 27.90         | 502              | OK            |
| 2026-03-10 12:00 | Zone_A  | 29.70             | 4537              | 15.90         | 442              | OK            |
| 2026-03-10 12:00 | Zone_B  | 36.00             | 4475              | 23.60         | 455              | OK            |
| 2026-03-10 12:00 | Zone_C  | 26.60             | 4276              | 25.40         | 493              | OK            |
| 2026-03-11 12:00 | Zone_A  | 30.10             | 4371              | 16.90         | 409              | OK            |
| 2026-03-11 12:00 | Zone_B  | 30.80             | 4280              | 20.70         | 471              | OK            |
| 2026-03-11 12:00 | Zone_C  | 26.90             | 4216              | 25.90         | 463              | OK            |
| 2026-03-12 12:00 | Zone_A  | 29.40             | 4384              | 19.80         | 434              | OK            |
| 2026-03-12 12:00 | Zone_B  | 26.50             | 4195              | 22.40         | 486              | OK            |
| 2026-03-12 12:00 | Zone_C  | 25.00             | 4100              | 23.40         | 499              | OK            |
| 2026-03-13 12:00 | Zone_A  | 27.90             | 4331              | 15.20         | 429              | OK            |
| 2026-03-13 12:00 | Zone_B  | 31.40             | 4234              | 22.60         | 477              | OK            |
| 2026-03-13 12:00 | Zone_C  | 27.30             | 4054              | 25.90         | 499              | OK            |
| 2026-03-14 12:00 | Zone_A  | 26.90             | 4246              | 17.50         | 396              | OK            |
| 2026-03-14 12:00 | Zone_B  | 30.00             | 4082              | 24.50         | 461              | OK            |
| 2026-03-14 12:00 | Zone_C  | 25.50             | 9900              | 25.50         | 493              | OK            |
| 2026-03-15 12:00 | Zone_A  | 26.70             | 4306              | 19.90         | 448              | OK            |
| 2026-03-15 12:00 | Zone_B  | 30.70             | 4137              | 20.70         | 447              | OK            |
| 2026-03-15 12:00 | Zone_C  | 25.30             | 4076              | 25.80         | 492              | OK            |
| 2026-03-16 12:00 | Zone_A  | 29.00             | 4269              | 18.90         | 401              | OK            |
| 2026-03-16 12:00 | Zone_B  | 30.60             | 4109              | 21.10         | 463              | OK            |
| 2026-03-16 12:00 | Zone_C  | 25.40             | 4040              | 25.20         | 489              | OK            |
| 2026-03-17 12:00 | Zone_A  | 25.90             | 4136              | 15.80         | 435              | OK            |
| 2026-03-17 12:00 | Zone_B  | 27.80             | 3961              | 22.30         | 452              | OK            |
| 2026-03-17 12:00 | Zone_C  | 24.70             | 3894              | 28.10         | 478              | OK            |
| 2026-03-18 12:00 | Zone_A  | 28.10             | 4217              | 18.70         | 422              | OK            |
| 2026-03-18 12:00 | Zone_B  | 28.60             | 3963              | 22.70         | 473              | OK            |
| 2026-03-18 12:00 | Zone_C  | 23.40             | 3902              | 25.10         | 523              | OK            |
| 2026-03-19 12:00 | Zone_A  | 27.90             | 4150              | 18.30         | 413              | OK            |
| 2026-03-19 12:00 | Zone_B  | 27.50             | 4074              | 23.50         | 463              | OK            |
| 2026-03-19 12:00 | Zone_C  | 24.50             | 4001              | 24.50         | 474              | OK            |
| 2026-03-20 12:00 | Zone_A  | 27.50             | 4078              | 18.70         | 422              | OK            |
| 2026-03-20 12:00 | Zone_B  | 27.90             | 3971              | 21.40         | 460              | OK            |
| 2026-03-20 12:00 | Zone_C  | 23.80             | 3920              | 27.50         | 483              | OK            |
| 2026-03-21 12:00 | Zone_A  | 27.10             | 4021              | 18.20         | 394              | OK            |
| 2026-03-21 12:00 | Zone_B  | 31.30             | 3875              | 0.00          | 468              | CHECK         |
| 2026-03-21 12:00 | Zone_C  | 21.20             | 3770              | 25.40         | 468              | OK            |
| 2026-03-22 12:00 | Zone_A  | 24.80             | 3863              | 15.90         | 434              | OK            |
| 2026-03-22 12:00 | Zone_B  | 24.30             | 3836              | 21.10         | 481              | OK            |
| 2026-03-22 12:00 | Zone_C  | 21.40             | 3686              | 26.40         | 487              | OK            |
| 2026-03-23 12:00 | Zone_A  | 24.50             | 3906              | 19.60         | 426              | OK            |
| 2026-03-23 12:00 | Zone_B  | 26.40             | 3824              | 20.70         | 490              | OK            |

# Appendices - Student Datasets for HydroSense-Kenya

| timestamp        | zone_id | soil_moisture_pct | tank_level_liters | pump_flow_lpm | pump_power_watts | sensor_status |
|------------------|---------|-------------------|-------------------|---------------|------------------|---------------|
| 2026-03-23 12:00 | Zone_C  | 22.60             | 3708              | 25.30         | 496              | OK            |
| 2026-03-24 12:00 | Zone_A  | 25.60             | 3765              | 18.30         | 419              | OK            |
| 2026-03-24 12:00 | Zone_B  | 25.80             | 3794              | 21.90         | 458              | OK            |
| 2026-03-24 12:00 | Zone_C  | 21.00             | 3699              | 25.10         | 467              | OK            |
| 2026-03-25 12:00 | Zone_A  | 25.00             | 3812              | 19.40         | 408              | OK            |
| 2026-03-25 12:00 | Zone_B  | 8.50              | 3690              | 21.00         | 442              | OK            |
| 2026-03-25 12:00 | Zone_C  | 21.60             | 3545              | 24.60         | 482              | OK            |
| 2026-03-26 12:00 | Zone_A  | 34.10             | 4146              | 18.40         | 432              | OK            |
| 2026-03-26 12:00 | Zone_B  | 32.60             | 4137              | 21.20         | 500              | OK            |
| 2026-03-26 12:00 | Zone_C  | 27.90             | 3944              | 25.00         | 508              | OK            |
| 2026-03-27 12:00 | Zone_A  | 25.10             | 3690              | 17.60         | 432              | OK            |
| 2026-03-27 12:00 | Zone_B  | 27.10             | 3641              | 18.80         | 454              | OK            |
| 2026-03-27 12:00 | Zone_C  | 19.60             | 3511              | 25.80         | 514              | OK            |
| 2026-03-28 12:00 | Zone_A  | 22.60             | 3697              | 17.00         | 411              | OK            |
| 2026-03-28 12:00 | Zone_B  | 23.70             | 3627              | 22.90         | 477              | OK            |
| 2026-03-28 12:00 | Zone_C  | 18.60             | 3481              | 26.20         | 502              | OK            |
| 2026-03-29 12:00 | Zone_A  | 23.30             | 3534              | 17.70         | 415              | OK            |
| 2026-03-29 12:00 | Zone_B  | 26.20             | 3565              | 24.60         | 441              | OK            |
| 2026-03-29 12:00 | Zone_C  | 18.90             | 3369              | 24.90         | 489              | OK            |
| 2026-03-30 12:00 | Zone_A  | 24.20             | 3512              | 20.40         | 415              | OK            |
| 2026-03-30 12:00 | Zone_B  | 23.80             | 3415              | 20.80         | 461              | OK            |
| 2026-03-30 12:00 | Zone_C  | 19.90             | 3442              | 27.20         | 502              | OK            |

## Appendix C: crop\_zone\_parameters.csv

| zone_id | crop_type | area_m2 | min_moisture_pct | target_moisture_pct | field_capacity_pct | drainage_coefficient |
|---------|-----------|---------|------------------|---------------------|--------------------|----------------------|
| Zone_A  | tomato    | 120     | 22               | 33                  | 41                 | 0.18                 |
| Zone_B  | kale      | 90      | 24               | 35                  | 43                 | 0.15                 |
| Zone_C  | maize     | 180     | 20               | 31                  | 40                 | 0.22                 |